```
  const { isSignedIn } = useUser();
  const router = useRouter();

  useEffect(() => {
    if (!isSignedIn) {.
      router.push('/sign-in');
    } else {
      router.push('/chat')
    }
  }, [isSignedIn, router]);

  return (
    <div>
        Loading...
    </div>
  );
}
```

The useUser hook from @clerk/nextjs gets the current user.

Imports the useRouter hook from next/navigation to perform redirects

Uses the isSignedIn property from the useUser hook to redirect to the sign-in page if the user is not authenticated

At this point, feel free to test out this interaction with the provided project. This is located at ch09/chat-authentication in the source code of this book:

```
$ npm install
$ npm run dev
```

Access the application at `http://localhost:3000`. After running the application, check that the flow works as expected:

- If you visit /, you should be redirected to the Clerk-hosted `/sign-in` page.
- After signing in, you should be redirected to the `/chat` page.

Now, with all the previous steps completed, the only task remaining is to set up a message quota limit for each authenticated user. In the next part, we'll walk through the steps to implement this quota using Upstash Redis.

> **NOTE** If you encounter a "New sign-ups are currently restricted" error when trying to sign in with Clerk, you will need to access your Clerk Dashboard at https://dashboard.clerk.com/, navigate to your application's settings, and adjust the Sign-Up Strategy to allow new users or add your specific email address to the list of allowed emails. For example, I added my Google email to the list of users, and then I was able to log in.

9.3.2 *Practical security control: Rate limiting*

Rate limiting is crucial for protecting APIs from abuse by setting a maximum threshold on the number of requests a client can make within a specified time. This is especially important when utilizing external LLMs, since they also have their own rate-limiting policies. To prevent your application from being rate limited or banned from those services, you must also limit the number of requests per second you send to the APIs.